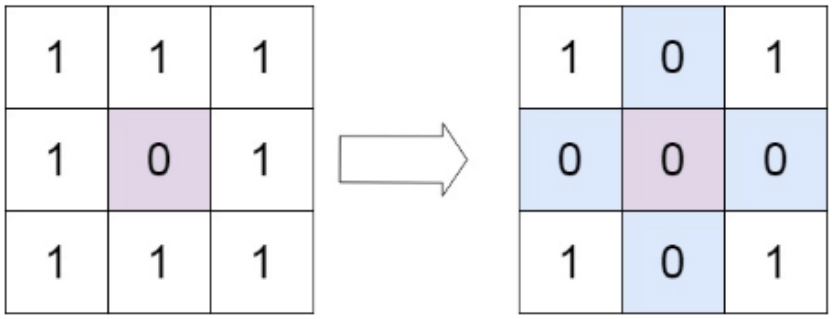
- 矩阵置零
  - 链接：矩阵置零
  - 描述

73. 矩阵置零

中等 相关标签 相关企业 提示 A文

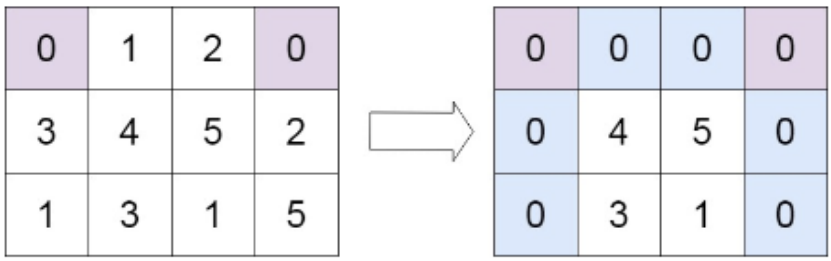
给定一个 `m x n` 的矩阵，如果一个元素为 **0** ，则将其所在行和列的所有元素都设为 **0** 。请使用 **原地** 算法。

示例 1：



> **输入：** `matrix = [[1,1,1],[1,0,1],[1,1,1]]`
> **输出：** `[[1,0,1],[0,0,0],[1,0,1]]`

示例 2：



> **输入：** `matrix = [[0,1,2,0],[3,4,5,2],[1,3,1,5]]`
> **输出：** `[[0,0,0,0],[0,4,5,0],[0,3,1,0]]`

提示：

- `m == matrix.length`
- `n == matrix[0].length`
- `1 <= m, n <= 200`
- `-2^31 <= matrix[i][j] <= 2^31 - 1`

  - 答案

```
function setZeroes(matrix: number[][]): void {
  const row = matrix.length
```

```
  const col = matrix[0].length
  const rowFalg = new Array(row).fill(false)
  const colFalg = new Array(col).fill(false)
  for (let i = 0; i < row; i++) {
    for (let j = 0; j < col; j++) {
      if (matrix[i][j] === 0) {
        rowFalg[i] = true
        colFalg[j] = true
      }
    }
  }
  for (let i = 0; i < row; i++) {
    for (let j = 0; j < col; j++) {
      if (rowFalg[i] || colFalg[j]) {
        matrix[i][j] = 0
      }
    }
  }
}
```

- 螺旋矩阵
  - 链接：螺旋矩阵
  - 描述

54. 螺旋矩阵

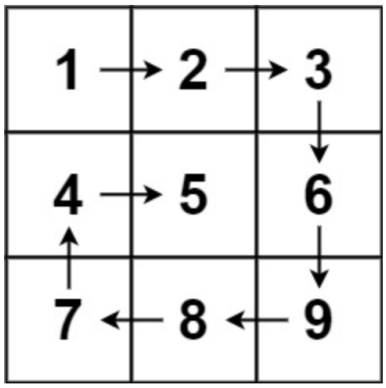
iShot (Purchase unlock to remove watermark)

已解答

中等 相关标签 相关企业 提示

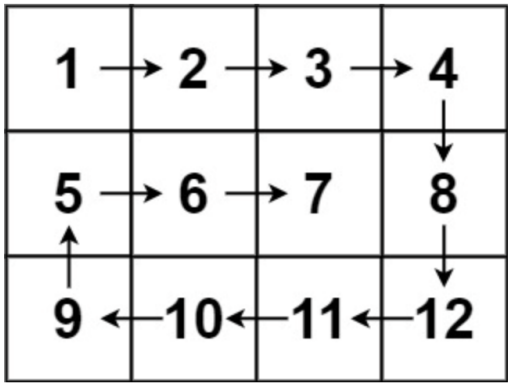
给你一个 `m` 行 `n` 列的矩阵 `matrix`，请按照 **顺时针螺旋顺序**，返回矩阵中的所有元素。

示例 1：



> **输入：** `matrix = [[1,2,3],[4,5,6],[7,8,9]]`
> **输出：** `[1,2,3,6,9,8,7,4,5]`

示例 2：



> **输入：** `matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]`
> **输出：** `[1,2,3,4,8,12,11,10,9,5,6,7]`

提示：

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 10`
- `-100 <= matrix[i][j] <= 100`

- 答案

```
function spiralOrder(matrix: number[][]): number[] {
  if (matrix.length === 0 || matrix[0].length === 0) {
    return []
  }
```

```
  const result: number[] = []
  let top = 0
  let bottom = matrix.length - 1
  let left = 0
  let right = matrix[0].length - 1
  while (top <= bottom && left <= right) {
    for (let i = left; i <= right; i++) {
      result.push(matrix[top][i])
    }
    top++

    for (let i = top; i <= bottom; i++) {
      result.push(matrix[i][right])
    }
    right--

    if (top <= bottom) {
      for (let i = right; i >= left; i--) {
        result.push(matrix[bottom][i])
      }
      bottom--
    }

    if (left <= right) {
      for (let i = bottom; i >= top; i--) {
        result.push(matrix[i][left])
      }
      left++
    }
  }
  return result
}
```